PROJET PIM
CODAGE DE HUFFMAN

Par Yan Rabefaniraka et Abel Fares

STRUCTURES DE DONNES

Arbre

```
type T_Noeud_Arbre;
type T_ARBRE is access T_Noeud_Arbre;

type T_Noeud_Arbre is record
   Cle: T_Cle;
   Valeur: T_Valeur;
   Fils_Gauche: T_ARBRE;
   Fils_Droit: T_ARBRE;
end record;
```

Table d'Arbre

```
type T_Cellule_Arbre;
type T_ptr_arbre is access T_Cellule_Arbre;
type T_Cellule_Arbre is record
   Arbre : T_ARBRE;
   Suivante : T_ptr_arbre;
end record;
```

STRUCTURES DE DONNES

Table de symboles

```
type T_Tab_symbole;
type T_ptr_symbole is access T_Tab_symbole;
type T_Tab_symbole is record
   Symbole : T_Cle;
   Suivante : T_ptr_symbole;
end record;
```

Table de parcour infixe

```
type T_Parcour_Infixe;
type T_ptr_infixe is access T_Parcour_Infixe;
type T_Parcour_Infixe is record
   Valeur: Integer;
   Suivante: T_ptr_infixe;
end record;
```

STRUCTURES DE DONNES

Table de Huffman

```
type T_Cellule_Huffman;
type T_ptr_Huffman is access T_Cellule_Huffman;
type T_Cellule_Huffman is record
   Symbole : T_Cle;
   Tab_encodage : T_ptr_infixe;
   Suivante : T_ptr_Huffman;
end record;
```

LCA

```
type T_Struct_LCA;
type T_LCA is access T_Struct_LCA;

type T_Struct_LCA is record
    Cle: T_Cle;
    Valeur: T_Valeur;
    Suivante: T_LCA;
end record;
```

STRUCTURES DE DONNES

Octet

```
type T_Octet is mod 2**8;
type T_Tab_Octet is array(File_Size range <>) of T_Octet;
type T_ptr_Tab_Octet is access T_Tab_Octet;
```

COMPRESSION

Gestion des arguments entrés en ligne de commande

```
Si il n'y a pas d'arguments
    Afficher une erreur
    Lever une exception Argument_Invalide_Exception
SinonSi il n'y a qu'un argument
   Mettre Bavard à True
Sinon
    Recuperer l'avant dernier argument responsable de la verbosité ou lever une erreur
Fin si

Recuperer le nom du fichier à décompresser
```

COMPRESSION

Lecture du fichier d’entrée et création de la table des fréquences

```
Open(File, In_File, To_String(File_Name));
Tant que le fichier n'est pas fini,
    Lire chaque caractère
    Ajouter 1 à sa fréquence dans la LCA
    Si le caractère n'est pas dans la LCA,
        Ajouter le caractère à la LCA avec une fréquence de 1
    Fin si
Fin tant que
```

COMPRESSION

Créer l’arbre de Huffman en utilisant la table d’arbres

```
Remplir la table d'arbres avec les feuilles générés à partir des fréquences de chaque symbole

Tant que la taille de la table d'arbres est supérieure à 1
    Trouver les deux arbres de poids minimum
    Supprimer ces deux arbres de la table d'arbres
    Créer un nouvel arbre avec ces deux arbres comme fils
    Ajouter le nouvel arbre à la table d'arbres
Fin Tant que

Afficher l'arbre restant dans la table d'arbres
```

COMPRESSION

Créer la table de parcours infixe et des symboles de l’arbre de Huffman

```
Si le noeud courant n'est pas une feuille
    Enregistrer 0 dans le parcours infixe
    continuer l'exploration pour le fils gauche
    continuer l'exploration pour le fils droit
sinon
    on enregistre 1 dans le parcours infixe
    On enregistre le symbole dans le tableau de symbole
Fin Si
```

COMPRESSION

Création et affichage de la table d’Huffman

```
Pour I dans 1..Taille(Table_symbole)
    récupérer le symbole à la Ieme position dans la table des symboles
    récupérer l'encodage du symbole dans l'arbre de Huffman
    enregistrer le symbole et l'encodage dans la table de Huffman
    Afficher le symbole et son encodage
Fin pour
```

COMPRESSION

Créer la table des entiers représentant les symboles en ASCII

```
Obtenir la position du caractère de fin

Pour I dans 1..Taille(Frequences)
    Si le symbole est différent du symbole de fin
        Si le symbole est "\n"
            enregistrer 10 représentant "\n" en ASCII
        Sinon
            enregistrer l'entier représentant le caractère en ASCII
        Fin si
Fin pour
enregistrer le dernier caractère pour le doubler à la fin de la table
```

COMPRESSION

Ecrire la table des symboles dans le fichier binaire

```
Ouvrir le nouveau fichier binaire

Pour I dans 1..Taille(Table des entiers représentant les symboles)
    convertir le Ième entier en Octet
    écrire l'octet dans le fichier
Fin pour
```

COMPRESSION

Ecrire le parcours infixe dans le fichier binaire

```
Ecrire le parcours infixe dans une chaine de caractère
mettre l'octet et le compte de bits à 0

Pour I dans 1..Taille(Table du parcours infixe)
    Ecrire le Ième bit dans l'octet
    Incrémenter le compte de bits
    Si le compte est à 8
        Ecrire l'octet dans le fichier
        remettre l'octet et le compte à 0
    Fin si
Fin pour

Si le compte est supérieur à 0
    remplir les derniers bits restant avec des 0
Fin si
```

COMPRESSION

Ecrire le contenu du fichier d’entrée dans le fichier binaire

```
Ouvrir le fichier d'origine

Tant que le fichier d'origine n'est pas fini
    Lire un caractère
    Trouver son encodage dans la table de Huffman
    l'ecrire dans un octet puis dans le fichier binaire
Fin Tant que

Ecrire le symbole de fin et compléter le dernier octet avec des 0 si besoin
```

DECOMPRESSION

Gestion des arguments entrés en ligne de commande

```
Si il n'y a pas d'arguments
    Afficher une erreur
    Lever une exception Argument_Invalide_Exception
SinonSi il n'y a qu'un argument
  Mettre Bavard à True
Sinon
    Recuperer l'avant dernier argument responsable de la verbosité ou lever une erreur
Fin si

Recuperer le nom du fichier à décompresser
Si le nom du fichier n'a pas l'extension ".hff"
  Afficher une erreur
  Lever une exception Argument_Invalide_Exception
Fin si
```

DECOMPRESSION

Lecture du fichier compressé

```
Lire le fichier compressé et stocker tous ses éléments dans un tableau d'octets
Obtenir la position du caractère de fin dans l'arbre de Huffman
```

DECOMPRESSION

Récupérer la liste des symboles

```
Recupérer et convertir en symboles les octets avant la position du symbole de fin
Ajourter le symbole de fin
Récupérer les octets après le symbole de fin et les convertir en symboles jusqu'à la répétition d'un octet
Afficher la liste des symboles
```

DECOMPRESSION

Recréer l’arbre de Huffman

```
Initialiser l'arbre
Recréer l'arbre de Huffman à partir de la table des symboles et du parcours infixe
Afficher l'arbre de Huffman si nécessaire
```

DECOMPRESSION

Ecriture du fichier décompressé

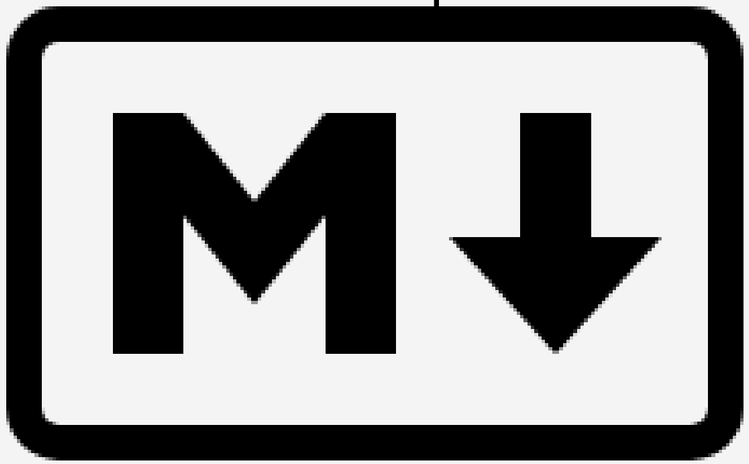
```
Ouvrir le fichier décompressé
Décoder le premier symbole
Ecrire le code binaire ASCII associé à ce symbole dans le fichier décompressé
Tant que le symbole est différent du symbole de File_Name
    Décoder le symbole suivant
    Ecrire le code binaire ASCII associé à ce symbole dans le fichier décompressé
Fin Tant que
Fermer le fichier décompressé
```

Outils utilisés

Gitlab



Markdown pandoc



Vscode



MERCI POUR VOTRE ECOUTE

COMPRESSION

Lecture du fichier d’entrée et création de la table des fréquences

```
-- Initialiser la liste des fréquences
Initialiser(Frequences);

-- Ouvrir le fichier
Open(File, In_File, To_String(File_Name));
Stream_input := Stream(File);

-- Lire chaque charactère et changer sa fréquence dans la LCA
while not End_Of_File(File) loop
   Ch := Character'Input(Stream_input);
   if Ch /= ASCII.CR then
      if Ch = ASCII.LF then
         begin
            Enregistrer(Frequences, "\n", La_Valeur(Frequences, "\n") + 1);
            exception
               when Cle_Absente_Exception =>
                  Enregistrer(Frequences, "\n", 1);
               when others =>
                  Null;
         end;

      else
         begin
            Enregistrer(Frequences, To_T_String(Ch), La_Valeur(Frequences, To_T_String(Ch)) + 1);
            exception
               when Cle_Absente_Exception =>
                  Enregistrer(Frequences, To_T_String(Ch), 1);
               when others =>
                  Null;
         end;
      end if;
   end if;
end loop;
Enregistrer(Frequences, "\$", 0);

-- Fermer le fichier
Close(File);
```

COMPRESSION

Créer l’arbre de Huffman en utilisant la table d’arbres

```
--Initialiser l'arbre vide pour créer des feuilles
AB_Initialiser(Arbre_vide);

-- Créer le tableau des feuilles
for i in 1..Taille(Frequences) loop
   AB_Creer_Noeud(Nouvel_Arbre, Cle_indice(Frequences, i), La_Valeur(Frequences, Cle_indice(Frequences, i)), Arbre_vide, Arbre_vide);
   Enregistrer_Arbre(Table_arbres, Nouvel_Arbre);
end loop;

-- Créer l'arbre de Huffman
while Taille_Table_Arbre(Table_arbres) > 1 loop
   Min(Table_arbres, Min1, Min2);
   if AB_La_Valeur(Min1) > AB_La_Valeur(Min2) then
      AB_Creer_Noeud(Nouvel_Arbre,To_T_String('0'), AB_La_Valeur(Min1) + AB_La_Valeur(Min2), Min2, Min1);
   else
      AB_Creer_Noeud(Nouvel_Arbre,To_T_String('0'), AB_La_Valeur(Min1) + AB_La_Valeur(Min2), Min1, Min2);
   end if;
   Enregistrer_Arbre(Table_arbres, Nouvel_Arbre);
end loop;
Assigner(Arbre_indice(Table_arbres, 1),Arbre_Huffman);
if Bavard then
   AB_Afficher(Arbre_Huffman);
   New_Line;
end if;

--Détruire le tableau d'arbre
Detruire_tab_arbre(Table_arbres);
```

COMPRESSION

```
procedure AB_Afficher_Debug(Arbre: in T_ARBRE) is
Taille : constant Integer := AB_Profondeur(Arbre);
type T_Tab is array(1..Taille) of Integer;
Tableau : T_Tab;
niveau : Constant Integer := 0;
   procedure Afficher(Arbre: in T_ARBRE; niveau: in Integer; Tab : in out T_Tab) is
   begin
      if AB_Est_Vide(Arbre) then
         Null;
      else
         if not AB_Est_Feuille(Arbre) then
            Afficher_Donnee(AB_La_Valeur(Arbre));
         else
            Afficher_Donnee(AB_La_Valeur(Arbre));
            Afficher_Cle(AB_La_Cle(Arbre));
         end if;
         if not AB_Est_Feuille(Arbre) then
            New_Line;
            if niveau /= 0 then
               for i in 1..niveau loop
                  if Tab(i) = 0 then
                     Put("|      ");
                  elsif Tab(i) = 1 then
                     Put("       ");
                  end if;
               end loop;
            end if;
            Put("\--0--");
         end if;
         Tab(niveau+1) := 0;
         Afficher(Arbre.all.Fils_Gauche, niveau + 1,Tab);
         if not AB_Est_Feuille(Arbre) then
            New_Line;
            if niveau /= 0 then
               for i in 1..niveau loop
                  if Tab(i) = 0 then
                     Put("|      ");
                  elsif Tab(i) = 1 then
                     Put("       ");
                  end if;
               end loop;
            end if;
            Put("\--1--");
         end if;
         Tab(niveau+1) := 1;
         Afficher(Arbre.all.Fils_Droit, niveau + 1,Tab);
      end if;
   end Afficher;
begin
   for i in 1..Taille loop
      Tableau(i) := 2;
   end loop;
   Afficher(Arbre, niveau, Tableau);
end AB_Afficher_Debug;
```

COMPRESSION

Créer la table de parcours infixe et des symboles de l’arbre de Huffman

```
-- Créer les tableaux de parcours infixe et de symbole de l'arbre d'Huffman
Init_infixe(Tab_arbre_infixe);
Init_symbole(Tab_symbole);
Table_symbole_infixe(Arbre_Huffman, Tab_symbole, Tab_arbre_infixe);
```

```
procedure Table_symbole_infixe(Arbre : in T_ARBRE; Tab_symbole : out T_ptr_symbole; Tab_arbre : out T_ptr_infixe) is
begin
   if Arbre.all.Fils_Gauche /= Null then
      Enregistrer_infixe(Tab_arbre, 0);
      Table_symbole_infixe(Arbre.all.Fils_Gauche, Tab_symbole, Tab_arbre);
      Table_symbole_infixe(Arbre.all.Fils_Droit, Tab_symbole, Tab_arbre);
   else
      Enregistrer_infixe(Tab_arbre, 1);
      Enregistrer_symbole(Tab_symbole, Arbre.all.Cle);
   end if;
end Table_symbole_infixe;
```

COMPRESSION

Création et affichage de la table d’Huffman

```
--Afficher la table de Huffmann
declare
   Bit_sequence : String(1..Taille_infixe(Tab_arbre_infixe));
   Tab : T_ptr_infixe;
   Tab_out : T_ptr_infixe;
   Symbole : T_String;
begin
   for I in 1..Taille_symbole(Tab_symbole) loop
      Symbole := Symbole_Indice(Tab_symbole, I);
      Symbole_encodage(Arbre_Huffman, Symbole, False, 0, Tab, Tab_out);
      Enregistrer_Tab_Huffman(Symbole, Tab_out, Table_Huffman);
      if Bavard then
         New_Line;
         if Symbole(2) = ' ' then
            Put("'" & Symbole(1) & "'" & Symbole(2));
         else
            Put("'" & Symbole(1) & Symbole(2) & "'");
         end if;
         Put(" --> ");
         for j in 1..Taille_infixe(Tab_out) loop
            Bit_sequence(j) := Character'Val(48 + Infixe_Indice(Tab_out, j));
            Put(Bit_sequence(j));
         end loop;
      end if;
   end loop;
end;
```

COMPRESSION

```
procedure Symbole_encodage(Arbre : in T_ARBRE; Symbole : in T_Cle; gauche : in Boolean; indice : in Integer; Tab : in out T_ptr_infixe; Tab_out : out T_ptr_infixe) is
Curseur : T_ptr_infixe :=Tab;
begin
   if indice=0 then
      Symbole_encodage(Arbre.all.Fils_Gauche, Symbole, True, indice+1, Tab, Tab_out);
      Symbole_encodage(Arbre.all.Fils_Droit, Symbole, False, indice+1, Tab, Tab_out);
   else
      --Effacer les valeurs de Tab ayant un indice supérieur à l'indice actuel
      if Tab/=Null then
         if indice = 1 and Tab.all.Suivante /= Null then
            Detruire_infixe(Tab.all.Suivante);
            Tab.all.Suivante := Null;
         elsif indice > 1 then
            for i in 1..indice-1 loop
               Curseur:=Curseur.all.Suivante;
            end loop;
            if Curseur/=Null then
               Detruire_infixe(Curseur.all.Suivante);
            end if;
         end if;
      end if;
      if gauche then
         Enregistrer_infixe_indice(Tab,0,indice);
      else
         Enregistrer_infixe_indice(Tab,1,indice);
      end if;
      if AB_Est_Feuille(Arbre) then
         if Arbre.all.Cle=Symbole then
            Assigner_Infixe(Tab, Tab_out);
         end if;
      else
         Symbole_encodage(Arbre.all.Fils_Gauche, Symbole, True, indice+1, Tab, Tab_out);
         Symbole_encodage(Arbre.all.Fils_Droit, Symbole, False, indice+1, Tab, Tab_out);
      end if;
   end if;
end Symbole_encodage;
```

COMPRESSION

Créer la table des entiers représentant les symboles en ASCII

```
--Créer la table des entiers représentant les symboles
Init_infixe(Tab_Int_Char);
Position_char_fin := Indice_symbole_fin(Tab_symbole);

Enregistrer_infixe(Tab_Int_Char, Position_char_fin);
for i in 1..Taille(Frequences) loop
   if Different_fin(Symbole_Indice(Tab_symbole, i)) then
      if Symbole_Indice(Tab_symbole, i)= "\n" then
         Enregistrer_infixe(Tab_Int_Char, 10);
      else
         Enregistrer_infixe(Tab_Int_Char, Character'Pos(To_Character(Symbole_Indice(Tab_symbole, i))));
      end if;
   end if;
end loop;
Enregistrer_infixe(Tab_Int_Char, Character'Pos(To_Character(Symbole_Indice(Tab_symbole, Taille(Frequences)))));
```

COMPRESSION

Ecrire la table des symboles dans le fichier binaire

```
-- Ouvrir un nouveau fichier avec l'extension .hff pour y écrire les données compressées
New_File_Name:= File_Name & ".hff";
Create(New_File, Out_File, To_String(New_File_Name));

-- Écrire les données compressées dans le nouveau fichier

-- Ecrire la liste des symboles dans l'ordre du parcours infixe en binaire
Stream_output := Stream(New_File);
for i in 1..Taille(Frequences)+1 loop
   Octet := T_Octet(Infixe_Indice(Tab_Int_Char, i));
   T_Octet'Write(Stream_output, Octet);
end loop;
```

COMPRESSION

Ecrire le parcours infixe dans le fichier binaire

```
declare
   Bit_sequence : String(1..Taille_infixe(Tab_arbre_infixe));
begin
   for i in Bit_sequence'Range loop
      Bit_sequence(i) := Character'Val(48 + Infixe_Indice(Tab_arbre_infixe, i));
   end loop;

   Octet:=0;

   -- Parcourir la séquence de bits
   for I in Bit_sequence'Range loop

      -- Ecrire les bits dans l'octet par ordre de poids
      if Bit_Sequence(I) = '1' then
         Octet := (Octet*2) or 1;
      else
         Octet := (Octet*2) or 0;
      end if;

      -- Augmenter le bit count permettant de savoir combien de bits ont été accumulés dans l'octet
      Bit_Count := Bit_Count + 1;

      -- Lorsque l'octet est complété écire l'octet dans le fichier
      if Bit_Count = 8 then
         T_Octet'Write(Stream_output, Octet);
         Octet := 0;
         Bit_Count := 0;
      end if;
   end loop;

   -- Remplir le dernier octet avec des zéros
   if Bit_Count > 0 then
      Octet := Shift_Left(Octet, 8 - Bit_Count);
      T_Octet'Write(Stream_output, Octet);
   end if;
end;
```

COMPRESSION

Ecrire le contenu du fichier d’entrée dans le fichier binaire

```
Open(File, In_File, To_String(File_Name));
Stream_input:= Stream(File);
Octet:=0;
Bit_Count:=0;
declare
   Bit_sequence : String(1..Taille_infixe(Tab_arbre_infixe));
   Symbole : T_String;
   Tab_encodage : T_ptr_infixe;
begin
   while not End_Of_File(File) loop
      Ch := Character'Input(Stream_input);
      if Ch /= ASCII.CR then
         if Ch = ASCII.LF then
            Symbole:= "\n";
         else
            Symbole := To_T_String(Ch);
         end if;
         Encodage_symbole(Symbole, Table_Huffman, Tab_encodage);
         for i in 1..Taille_infixe(Tab_encodage) loop
            Bit_sequence(i) := Character'Val(48 + Infixe_Indice(Tab_encodage, i));
         end loop;
         -- Parcourir la séquence de bits
         for I in 1..Taille_infixe(Tab_encodage) loop

            -- Ecrire les bits dans l'octet par ordre de poids
            if Bit_Sequence(I) = '1' then
               Octet := (Octet*2) or 1;
            else
               Octet := (Octet*2) or 0;
            end if;

            -- Augmenter le bit count permettant de savoir combien de bits ont été accumulés dans l'octet
            Bit_Count := Bit_Count + 1;

            -- Lorsque l'octet est complété écire l'octet dans le fichier
            if Bit_Count = 8 then
               T_Octet'Write(Stream_output, Octet);
               Octet := 0;
               Bit_Count := 0;
            end if;
         end loop;
      end if;
   end loop;
```

COMPRESSION

```
      -- Ajouter le symbole de fin
      Symbole := "\$";
      Encodage_symbole(Symbole, Table_Huffman, Tab_encodage);
      for i in 1..Taille_infixe(Tab_encodage) loop
         Bit_sequence(i) := Character'Val(48 + Infixe_Indice(Tab_encodage, i));
      end loop;
      for I in 1..Taille_infixe(Tab_encodage) loop

         -- Ecrire les bits dans l'octet par ordre de poids
         if Bit_Sequence(I) = '1' then
            Octet := (Octet*2) or 1;
         else
            Octet := (Octet*2) or 0;
         end if;

         -- Augmenter le bit count permettant de savoir combien de bits ont été accumulés dans l'octet
         Bit_Count := Bit_Count + 1;

         -- Lorsque l'octet est complété écire l'octet dans le fichier
         if Bit_Count = 8 then
            T_Octet'Write(Stream_output, Octet);
            Octet := 0;
            Bit_Count := 0;
         end if;
      end loop;
      -- Remplir le dernier octet avec des zéros
      if Bit_Count > 0 then
         Octet := Shift_Left(Octet, 8 - Bit_Count);
         T_Octet'Write(Stream_output, Octet);
      end if;
      --Detruire les tables infixe obsolètes
      Detruire_infixe(Tab_encodage);
   end;

   --Detruire les tables infixe obsolètes
   Detruire_infixe(Tab_arbre_infixe);
   Detruire_infixe(Tab_Int_Char);
   --Detruire les tables infixe obsolètes
   Detruire_symbole(Tab_symbole);
   --Detruire Arbres obslètes
   AB_Detruire(Arbre_Huffman);
   --Detruire la table de Huffman
   Detruire_tab_Huffman(Table_Huffman);

   -- Fermer le fichier input
   Close(File);

   -- Fermer le fichier output
   Close(New_File);

   --Détruire la liste des fréquences
   Detruire(Frequences);
end Compression;
```

DECOMPRESSION

Gestion des arguments entrés en ligne de commande

```
begin
  --Vérifier que le nombre d'arguments est correct
   if Ada.Command_Line.Argument_Count < 1 then
      Put_Line("Utilisation: ./decompression <file_name>. Manque d'arguments.");
      raise Argument_Invalide_Exception;

   --Vérifier les arguments et établir le mode
   elsif Ada.Command_Line.Argument_Count = 1 then
      Bavard := True;

   else
      if Ada.Command_Line.Argument (Ada.Command_Line.Argument_Count-1) = "-b" or
      Ada.Command_Line.Argument (Ada.Command_Line.Argument_Count-1) = "--bavard" then
         Bavard := True;

      elsif Ada.Command_Line.Argument (Ada.Command_Line.Argument_Count-1) = "-s" or
      Ada.Command_Line.Argument (Ada.Command_Line.Argument_Count-1) = "--silencieux" then
         Bavard := False;

      else
         Put_Line("Utilisation: ./decompression [-b | --bavard | -s | --silencieux] <file_name>.");
         raise Argument_Invalide_Exception;
      end if;
   end if;

   -- Récupérer le nom du fichier à décompresser
   File_Name := To_Unbounded_String(Ada.Command_Line.Argument(Ada.Command_Line.Argument_Count));
   if To_String(File_Name)'Length < 3 then
      Put_Line("Erreur: Le fichier n'est pas un fichier compressé de Huffman.");
      raise Argument_Invalide_Exception;
   else
      if not Est_Hff(To_String(File_Name)) then
         Put_Line("Erreur: Le fichier n'est pas un fichier compressé de Huffman.");
         raise Argument_Invalide_Exception;
      end if;
   end if;
```

DECOMPRESSION

Lecture du fichier compressé

```
-- Ouvrir le fichier à décompresser
Tab_Entree := Lire_Fichier_Bin(To_String(File_Name));
Position_char_fin := Integer(Tab_Entree.all(1));
```

```
--Lire le fichier binaire
function Lire_Fichier_Bin(Filename : String) return T_ptr_Tab_Octet is

   Taille_Fichier_Bin : File_Size := Ada.Directories.Size(Filename);
   Contenu : T_ptr_Tab_Octet;
   Flux : Stream_Access;
   File : File_Type;

begin
   Contenu := new T_Tab_Octet (1 .. Taille_Fichier_Bin);
   Open(File, In_File, Filename);
   Flux := Stream(File);

   T_Tab_Octet'Read (Flux, Contenu.all);

   Close (File);

   return Contenu;
end Lire_Fichier_Bin;
```

DECOMPRESSION

```
--Créer Tableau de symboles (on les gardera en binaire dans la version finale)
LCA_S.Initialiser(Tab_Symbole_Binaire_Original);


--Ajouter les symboles avant le caractère de fin
for i in 2..Position_char_fin+1 loop
   Octet := Tab_Entree.all(File_Size(i));
   if Octet = 10 then
      LCA_S.Enregistrer(Tab_Symbole_Binaire_Original, "\n", Octet);
   else
      Tmp_Char := Character'Val(Octet);
      LCA_S.Enregistrer(Tab_Symbole_Binaire_Original, To_T_String(Tmp_Char), Octet);
   end if;
end loop;

--Ajouter le symbole de fin
Tmp_String := ('\', '$');
LCA_S.Enregistrer(Tab_Symbole_Binaire_Original, Tmp_String, -1);
Tmp_Exploration_Index := Position_char_fin + 2;
Tmp_Octet := Tab_Entree.all(File_Size(Tmp_Exploration_Index+1));


--Ajouter les symboles après le caractère de fin jusqu'à répétitition d'un octet.
Octet := Tab_Entree.all(File_Size(Tmp_Exploration_Index));
while not Est_Meme_Octet(Octet, Tmp_Octet) loop
   if Octet = 10 then
      LCA_S.Enregistrer(Tab_Symbole_Binaire_Original, "\n", Octet);
   else
      Tmp_Char := Character'Val(Octet);
      LCA_S.Enregistrer(Tab_Symbole_Binaire_Original, To_T_String(Tmp_Char), Octet);
   end if;
   Tmp_Exploration_Index := Tmp_Exploration_Index + 1;
   Octet := Tab_Entree.all(File_Size(Tmp_Exploration_Index));
   Tmp_Octet := Tab_Entree.all(File_Size(Tmp_Exploration_Index+1));
end loop;
Tmp_Char := Character'Val(Octet);
LCA_S.Enregistrer(Tab_Symbole_Binaire_Original, To_T_String(Tmp_Char), Octet);


--Afficher les symboles récupérés
if Bavard then
   New_Line;
   Put_Line("Tableau des symboles : ");
   Afficher_Debug(Tab_Symbole_Binaire_Original);
   New_Line;
end if;
```

DECOMPRESSION

Création de l’arbre de Huffman

```
--Récupérer la structure de l'arbre d'Huffman
   Tmp_Exploration_Index := Tmp_Exploration_Index + 2;
   AB_Creer_Noeud(Arbre_Huffman, To_T_String('0'), 0, Arbre_Vide, Arbre_Vide);

   --Récupérer le nombre de symboles
   Nb_Symbole := LCA_S.Taille(Tab_Symbole_Binaire_Original);
   Indice_Symboles := 1;
   Indice_Bit := 1;

   --Générer l'arbre de Huffman récursivement
   Generer_Arbre_Parcours(Tab_Symbole_Binaire_Original, Tab_Entree, Arbre_Huffman, Arbre_Vide,
   Indice_Symboles, Indice_Bit, Tmp_Exploration_Index, Nb_Symbole, To_T_String('0'), 0);


--Afficher l'arbre d'Huffman
   if Bavard then
      New_Line;
      Put_Line("Arbre d'Huffman : ");
      Afficher_Arbre(Arbre_Huffman);
      New_Line;
      Put_Line ("Index d'exploration et bit courants:" & Tmp_Exploration_Index'Image & "; " & Indice_Bit'Image);
   end if;
```

DECOMPRESSION

```
--Générer l'arbre de Huffman à partir du parcours infixe
procedure Generer_Arbre_Parcours(Tab_symbole : in T_LCA; Tab_Entree : in T_ptr_Tab_Octet;
Arbre, Arbre_vide : in out T_ARBRE; Indice_Symboles, Indice_Bit, Indice_Parcours_Entree, Indice_Symboles_Max : in out Integer;
Cle_Default : in T_Cle; Valeur_Default: in T_Valeur) is
   Bit : T_Octet;
   Octet : T_Octet := Tab_Entree.all(File_Size(Indice_Parcours_Entree));
begin
   --Récupérer le bit de poids fort
   Bit := Octet / 128;
   --Décaler l'octet de 1 bit vers la gauche
   Tab_Entree.all(File_Size(Indice_Parcours_Entree)) := Octet*2;
   --Générer l'arbre à partir du parcours infixe
   if Indice_Symboles <= Indice_Symboles_Max then
      --Avancer vers le prochain octet si on a parcouru tous les bits de l'octet actuel
      if Indice_Bit = 8 then
            Indice_Bit := 1;
            Indice_Parcours_Entree := Indice_Parcours_Entree + 1;
         else
            Indice_Bit := Indice_Bit + 1;
         end if;
      --Créer une feuille si on a un bit à 1
      if Bit = 1 then
         AB_Assigner_Cle(Arbre, LCA_S.Cle_indice(Tab_symbole, Indice_Symboles));
         AB_Assigner_Valeur(Arbre, LCA_S.La_Valeur(Tab_symbole, LCA_S.Cle_indice(Tab_symbole, Indice_Symboles)));
         Indice_Symboles := Indice_Symboles + 1;
      --Générer le sous-arbre gauche puis le sous-arbre droit à partir du parcours infixe
      else
         AB_Creer_Noeud(Arbre.all.Fils_Gauche, Cle_Default, Valeur_Default, Arbre_vide, Arbre_vide);
         AB_Creer_Noeud(Arbre.all.Fils_Droit, Cle_Default, Valeur_Default, Arbre_vide, Arbre_vide);
         Generer_Arbre_Parcours(Tab_symbole, Tab_Entree, Arbre.all.Fils_Gauche,
         Arbre_vide, Indice_Symboles, Indice_Bit, Indice_Parcours_Entree, Indice_Symboles_Max,
         Cle_Default, Valeur_Default);
         Generer_Arbre_Parcours(Tab_symbole, Tab_Entree, Arbre.all.Fils_Droit,
         Arbre_vide, Indice_Symboles, Indice_Bit, Indice_Parcours_Entree, Indice_Symboles_Max,
         Cle_Default, Valeur_Default);
      end if;
   end if;
end Generer_Arbre_Parcours;
```

DECOMPRESSION

Ecriture du fichier décompressé

```
--Créer le fichier décompressé
New_File_Name := File_Name & ".d";
Create(New_File, Out_File, To_String(New_File_Name));

--Décoder le texte compressé
   Tmp_Exploration_Index := Tmp_Exploration_Index + 1;
   Indice_Bit := 1;
   --Décoder le symbole codé courant à l'aide de l'arbre
   Decoder_Symbole_Compresse(Arbre_Huffman, Tab_Entree, Indice_Bit, Tmp_Exploration_Index, Tmp_String, Tmp_Octet);
   if Bavard then
      Put_Line("Symboles décodés : ");
      Afficher_Elem (Tmp_String);
   end if;
   --Ajouter le caractère dans le texte décompressé à afficher
   Og_Text := To_UB_String(Tmp_String);
   --Ecrire l'octet dans le fichier
   Write(New_File, Tmp_Octet);
   --Récupérer les symboles jusqu'à la fin du texte
   while Different_Fin(Tmp_String) loop
      --Décoder le symbole codé courant à l'aide de l'arbre
      Decoder_Symbole_Compresse(Arbre_Huffman, Tab_Entree, Indice_Bit, Tmp_Exploration_Index, Tmp_String, Tmp_Octet);
      --Afficher le symbole décodé associé à l'octet
      if Bavard then
         Afficher_Elem(Tmp_String);
      end if;
      --Ajouter le caractère dans le texte décompressé à afficher
      if Different_Fin(Tmp_String) then
         Og_Text := Og_Text & To_UB_String(Tmp_String);
         --Ecrire l'octet dans le fichier
         Write(New_File, Tmp_Octet);
      end if;
   end loop;

--Fermer le fichier après écriture
Close(New_File);
```

DECOMPRESSION

```
--Décoder un symbole compressé à partir de l'arbre de Huffman
procedure Decoder_Symbole_Compresse(Arbre: in T_ARBRE; Tab_Entree : in T_ptr_Tab_Octet; Indice_Bit, Indice_Parcours_Entree : in out Integer;
String: in out T_Cle; Octet_du_String: in out T_Valeur)
is
   Bit : T_Octet;
   Octet : T_Octet := Tab_Entree.all(File_Size(Indice_Parcours_Entree));

begin
   --  New_Line;
   --  Put_Line("Octet actuel avant changement : ");
   --  Put(Integer(Octet), Base => 2);
   --  New_Line;
   if AB_Est_Feuille(Arbre) then
      String := AB_La_Cle(Arbre);
      Octet_du_String := AB_La_Valeur(Arbre);
   else
      Bit := Octet / 128;
      Tab_Entree.all(File_Size(Indice_Parcours_Entree)) := Octet*2;
      if Indice_Bit = 8 then
         Indice_Bit := 1;
         Indice_Parcours_Entree := Indice_Parcours_Entree + 1;
      else
         Indice_Bit := Indice_Bit + 1;
      end if;
      if Bit = 1 then
         Decoder_Symbole_Compresse(Arbre.all.Fils_Droit, Tab_Entree, Indice_Bit, Indice_Parcours_Entree, String, Octet_du_String);
      else
         Decoder_Symbole_Compresse(Arbre.all.Fils_Gauche, Tab_Entree, Indice_Bit, Indice_Parcours_Entree, String, Octet_du_String);
      end if;
   end if;
end Decoder_Symbole_Compresse;
```